

Text as Data
ifo Institute, 15–17 September 2026
Benjamin W. Arolt, University of Cambridge

7. Foundations for LLMs: Word Embeddings and Neural Networks

Outline

Words Embedding with Local Contexts

Properties of Word Embeddings

How do I use Word Embeddings?

Neural Networks

Bias in Language

Outline

Words Embedding with Local Contexts

Properties of Word Embeddings

How do I use Word Embeddings?

Neural Networks

Bias in Language

Word Embeddings

- ▶ “Word embeddings” often refer to GloVe – these are particular (popular) models for producing word embeddings.
 - ▶ **Previously:** focus on global document counts to predict an outcome
 - ▶ **Now:** represent the meaning of words by the neighboring words – their **local contexts**.

Word Embeddings

- ▶ “Word embeddings” often refer to GloVe – these are particular (popular) models for producing word embeddings.
 - ▶ **Previously:** focus on global document counts to predict an outcome
 - ▶ **Now:** represent the meaning of words by the neighboring words – their **local contexts**.
 - rather than predicting some metadata, they predict the co-occurrence of neighboring words.

Word Embeddings

- ▶ “Word embeddings” often refer to GloVe – these are particular (popular) models for producing word embeddings.
 - ▶ **Previously:** focus on global document counts to predict an outcome
 - ▶ **Now:** represent the meaning of words by the neighboring words – their **local contexts**.
 - rather than predicting some metadata, they predict the co-occurrence of neighboring words.
 - ▶ “You shall know a word by the company it keeps”
- ▶ From high-dimensional sparse representations to low-dimensional dense representations

Words and Contexts

A long line of NLP research aims to capture the distributional properties of words using a **word-context matrix M** :

- ▶ each row w represents a **word** of the vocabulary (e.g. “income”), each column c represents another word (e.g. “tax”) or linguistic **context** in which words can occur (e.g. “... pay corporate ___ tax to the relevant ...”)
 - ▶ A matrix entry $M_{[w,c]}$ quantifies the strength of association between a word and a context in a large corpus.
 - ▶ Popular embeddings (such as glove) generally use 5- or 10-word context windows

Words and Contexts

A long line of NLP research aims to capture the distributional properties of words using a **word-context matrix** M :

- ▶ each row w represents a **word** of the vocabulary (e.g. “income”), each column c represents another word (e.g. “tax”) or linguistic **context** in which words can occur (e.g. “... pay corporate ___ tax to the relevant ...”)
 - ▶ A matrix entry $M_{[w,c]}$ quantifies the strength of association between a word and a context in a large corpus.
 - ▶ Popular embeddings (such as glove) generally use 5- or 10-word context windows
- ▶ each word (row) $M_{[w,:]}$ gives a distribution over contexts.
 - ▶ different definitions of contexts and different measures of association → different types of **word vectors**.
 - ▶ these vectors have a **spatial interpretation** → geometric distances between word vectors reflect semantic distances between words.

GloVe Embeddings

- ▶ Pennington et al (2014) GloVe (Global Vectors for Word Representation) is embedding technique to represent words as dense vectors to capture semantic relationships between words
- ▶ Input: C_{ij} = local co-occurrence counts between words $i, j \in \{1, \dots, n_w\}$ within some co-occurrence window, e.g. ten words.

GloVe Embeddings

- ▶ Pennington et al (2014) GloVe (Global Vectors for Word Representation) is embedding technique to represent words as dense vectors to capture semantic relationships between words
- ▶ Input: C_{ij} = local co-occurrence counts between words $i, j \in \{1, \dots, n_w\}$ within some co-occurrence window, e.g. ten words.

Learn word vectors $\mathbf{w} = (w_1, \dots, w_i, \dots, w_{n_w})$, where $w_i \in (-1, 1)^{n_E}$, to solve

$$\min_{\mathbf{w}} \sum_{i,j} f(C_{ij}) \left(w_i^T w_j - \log(C_{ij}) \right)^2$$

where $f(\cdot)$ is weighting function to down-weight frequent words.

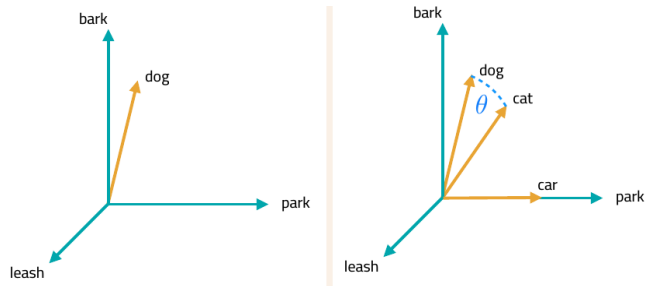
- ▶ Minimizes **squared difference** between:
 - ▶ **dot product of word vectors**, $w_i^T w_j$
 - ▶ **empirical co-occurrence**, $\log(C_{ij})$
[Arora et al (2016) put the PMI here instead of co-occurrence counts]
- ▶ Intuitively: words that co-occur should have high correlation (dot product)

Word2vec: From Counting to Predicting

- ▶ Like GloVe, word2vec starts from a simple idea: words appearing in similar contexts should have similar vectors.
- ▶ The main difference is how the vectors are learned:
 - ▶ **GloVe** uses a table counting how often words occur together.
 - ▶ **Word2vec** learns through a word-prediction task.
- ▶ Word2vec can use either of two prediction tasks:
 - ▶ **CBOW**: predict a word from the words around it.
 - ▶ **Skip-gram**: predict surrounding words from a given word.
- ▶ The model improves its word vectors whenever its predictions are wrong.
- ▶ After seeing many examples, words used in similar ways—such as *teacher* and *professor*—have similar vectors.

Word Similarity

- ▶ Once words are represented as vectors $\{v_1 = \mathbf{M}_{[w_1,:]}, v_2 = \mathbf{M}_{[w_2,:]}, \dots\}$, we can use linear algebra to understand the relationships between words:
 - ▶ Words that are geometrically close to each other are similar: e.g. “dog” and “cat”:



- ▶ The standard metric for comparing vectors is cosine similarity:

$$\cos \theta = \frac{v_1 \cdot v_2}{||v_1|| ||v_2||}$$

- ▶ Thanks to linearity, can compute similarities between groups of words by averaging the groups.

Word Vectors can produce Document Vectors

$$\vec{D} = \sum_{w \in D} a_w \vec{w}$$

- ▶ The “continuous bag of words” representation for document D is the sum, or the average (potentially weighted by a_w), of the vectors $\vec{w}$ for each word w in the document.
 - ▶ word vectors $\vec{w}$ constructed using GloVe (pre-trained or trained on the corpus).
 - ▶ “Document” could be sentence, paragraph, section, etc.

Word Vectors can produce Document Vectors

$$\vec{D} = \sum_{w \in D} a_w \vec{w}$$

- ▶ The “continuous bag of words” representation for document D is the sum, or the average (potentially weighted by a_w), of the vectors $\vec{w}$ for each word w in the document.
 - ▶ word vectors $\vec{w}$ constructed using GloVe (pre-trained or trained on the corpus).
 - ▶ “Document” could be sentence, paragraph, section, etc.
- ▶ Arora, Liang, and Ma (2017) provide a “tough to beat baseline”, the SIF-weighted (“smoothed inverse frequency”) average of the vectors:

$$a_w = \frac{\alpha}{\alpha + p_w}$$

where p_w is the probability (frequency) of the word and $\alpha = .001$ is a smoothing parameter.

What do Word Embeddings Capture? (Budansky and Hirst, 2006)

- ▶ Semantic **similarity**: words sharing salient attributes / features
 - ▶ synonymy (car / automobile)
 - ▶ hypernymy (car / vehicle)
 - ▶ co-hyponymy (car / van / truck)

What do Word Embeddings Capture? (Budansky and Hirst, 2006)

- ▶ Semantic **similarity**: words sharing salient attributes / features
 - ▶ synonymy (car / automobile)
 - ▶ hypernymy (car / vehicle)
 - ▶ co-hyponymy (car / van / truck)
- ▶ Semantic **relatedness**: words semantically associated without necessarily being similar
 - ▶ function (car / drive)
 - ▶ meronymy (car / tire)
 - ▶ location (car / road)
 - ▶ attribute (car / fast)

What do Word Embeddings Capture? (Budansky and Hirst, 2006)

- ▶ Semantic **similarity**: words sharing salient attributes / features
 - ▶ synonymy (car / automobile)
 - ▶ hypernymy (car / vehicle)
 - ▶ co-hyponymy (car / van / truck)
- ▶ Semantic **relatedness**: words semantically associated without necessarily being similar
 - ▶ function (car / drive)
 - ▶ meronymy (car / tire)
 - ▶ location (car / road)
 - ▶ attribute (car / fast)
- ▶ Word embeddings will recover one or both of these relations, depending on how contexts and vectors are constructed.

Most similar words to dog, depending on context window size

	2-word window	30-word window	
More paradigmatic		<u>kennel</u>	More syntagmatic
	cat	puppy	
	horse	pet	
	fox	bitch	
	pet	terrier	
	rabbit	rottweiler	
	pig	canine	
	animal	cat	
	mongrel	<u>bark</u>	
	sheep	alsatian	
	pigeon		

- ▶ Small windows pick up substitutable words; large windows pick up topics.

Parts of Speech and Phrases

- ▶ In the default model multiple senses of a word are merged.
 - ▶ e.g. “I like a bird” (verb) and “I am like a bird” (preposition).

Parts of Speech and Phrases

- ▶ In the default model multiple senses of a word are merged.
 - ▶ e.g. “I like a bird” (verb) and “I am like a bird” (preposition).
- ▶ Can improve the quality of embeddings in these cases by attaching the POS to the word (e.g. “like:verb”, “like:prep”) before training.

Parts of Speech and Phrases

- ▶ In the default model multiple senses of a word are merged.
 - ▶ e.g. “I like a bird” (verb) and “I am like a bird” (preposition).
- ▶ Can improve the quality of embeddings in these cases by attaching the POS to the word (e.g. “like:verb”, “like:prep”) before training.
- ▶ The default model only works by word, but “new york $\neq$ ”new” + “york”
 - ▶ can tokenize phrases together (see Week 2 lecture) before training.

The black sheep problem

- ▶ The trivial or obvious features of a word are not mentioned in standard corpora.

The black sheep problem

- ▶ The trivial or obvious features of a word are not mentioned in standard corpora.
- ▶ For example, although most sheep are white, you rarely see the phrase “white sheep”.
 - ▶ so glove tells you $\text{sim}(\text{black}, \text{sheep}) > \text{sim}(\text{white}, \text{sheep})$.

The black sheep problem

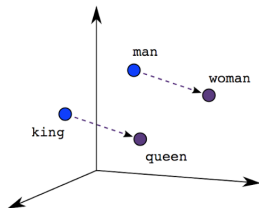
- ▶ The trivial or obvious features of a word are not mentioned in standard corpora.
- ▶ For example, although most sheep are white, you rarely see the phrase “white sheep”.
 - ▶ so glove tells you $\text{sim}(\text{black}, \text{sheep}) > \text{sim}(\text{white}, \text{sheep})$.
- ▶ This is really important when we will use embeddings to analyze beliefs/attitudes.
 - ▶ And I don't see a solution to it.

The black sheep problem

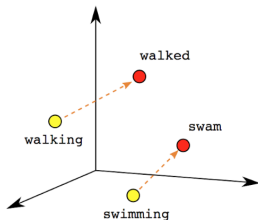
- ▶ The trivial or obvious features of a word are not mentioned in standard corpora.
- ▶ For example, although most sheep are white, you rarely see the phrase “white sheep”.
 - ▶ so glove tells you $\text{sim}(\text{black}, \text{sheep}) > \text{sim}(\text{white}, \text{sheep})$.
- ▶ This is really important when we will use embeddings to analyze beliefs/attitudes.
 - ▶ And I don't see a solution to it.
- ▶ Relatedly, antonyms are often rated similarly, have to be careful with that.

Vector Directions $\leftrightarrow$ Meaning

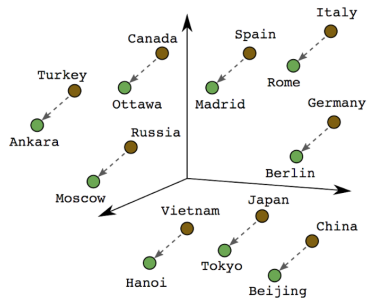
- ▶ Intriguingly, glove algebra can depict conceptual, analogical relationships between words:



Male-Female



Verb Tense



Country-Capital

Word Embeddings for Analogies

$$\text{vec}(\textit{king}) - \text{vec}(\textit{man}) + \text{vec}(\textit{woman}) \approx \text{vec}(\textit{queen})$$

Word Embeddings for Analogies

$$\text{vec}(\textit{king}) - \text{vec}(\textit{man}) + \text{vec}(\textit{woman}) \approx \text{vec}(\textit{queen})$$

- More generally: The analogy $a_1 : b_1 :: a_2 : b_2$ can be solved (that is, find b_2 given a_1, b_1, a_2) by

$$\arg \max_{b_2 \in V} \cos(b_2, a_2 - a_1 + b_1)$$

where V excludes (a_1, b_1, a_2) .

Word Embeddings for Analogies

$$\text{vec}(\textit{king}) - \text{vec}(\textit{man}) + \text{vec}(\textit{woman}) \approx \text{vec}(\textit{queen})$$

- More generally: The analogy $a_1 : b_1 :: a_2 : b_2$ can be solved (that is, find b_2 given a_1, b_1, a_2) by

$$\arg \max_{b_2 \in V} \cos(b_2, a_2 - a_1 + b_1)$$

where V excludes (a_1, b_1, a_2) .

- Often works better with normalized vectors (so that one long vector doesn't wash out the others)

Outline

Words Embedding with Local Contexts

Properties of Word Embeddings

How do I use Word Embeddings?

Neural Networks

Bias in Language

Tokenizing for Word Embeddings

- ▶ Need to think about what's the right way to pre-process data
 - ▶ drop capitalization
 - ▶ punctuation is optional
 - ▶ add special tokens for start of sentence and end of sentence
 - ▶ don't drop stopwords/function-words
 - ▶ for out-of-vocab words, substitute a special token or replace with part-of-speech tag

Which word-embedding?

- ▶ In many settings (e.g. a small corpus), better to use pre-trained embeddings.
- ▶ e.g, spaCy's GloVe embeddings:
 - ▶ one million vocabulary entries
 - ▶ 300-dimensional vectors
 - ▶ trained on the Common Crawl corpus
- ▶ Can initialize models with pre-trained embeddings, can fine-tune as needed.

“Enriching word vectors with subword information” (Bojanowski et al 2017)

- ▶ each word is represented as a bag of character n-grams. (e.g., spicy = (spi, pic, icy)).
- ▶ learn embeddings for the character segments, and construct word embedding by summing over the segment embeddings

“Enriching word vectors with subword information” (Bojanowski et al 2017)

- ▶ each word is represented as a bag of character n-grams. (e.g., spicy = (spi, pic, icy)).
- ▶ learn embeddings for the character segments, and construct word embedding by summing over the segment embeddings
- ▶ competitive with glove in standard tasks; better in some languages.

Limitations

Standard word embeddings (e.g. glove) have a number of limitations:

- ▶ **n-grams**: does not produce embeddings for multi-word phrases
- ▶ **rare words**: a word that shows up just once or twice won't be well-defined
- ▶ **polysemy**: you get one vector for multiple senses of a word
(e.g. “**glass** of water” vs “window **glass**”)

Outline

Words Embedding with Local Contexts

Properties of Word Embeddings

How do I use Word Embeddings?

Neural Networks

Bias in Language

From word2vec to neural networks

- ▶ word2vec learns embeddings by **predicting** a word from its context (or vice versa)
 - and the predictor it trains is a small **neural network**: one hidden layer whose weights *are* the word vectors

From word2vec to neural networks

- ▶ word2vec learns embeddings by **predicting** a word from its context (or vice versa)
 - and the predictor it trains is a small **neural network**: one hidden layer whose weights *are* the word vectors
- ▶ The same building block – layers of weighted sums passed through a nonlinearity, trained by gradient descent (Lecture 5) – scales from word2vec to today's large language models

From word2vec to neural networks

- ▶ word2vec learns embeddings by **predicting** a word from its context (or vice versa) – and the predictor it trains is a small **neural network**: one hidden layer whose weights *are* the word vectors
- ▶ The same building block – layers of weighted sums passed through a nonlinearity, trained by gradient descent (Lecture 5) – scales from word2vec to today's large language models
- ▶ Next: what a neural network is, how it is trained, and what its hyperparameters mean – the last prerequisite before Lecture 8 (LLMs and transformers)

Neural Nets: Basics

- ▶ Neural networks $\leftrightarrow$ deep learning models
 - ▶ solve machine learning problems, just like logistic regression or gradient boosted machines
 - ▶ use tensorflow, torch, or huggingface, rather than sklearn or xgboost.

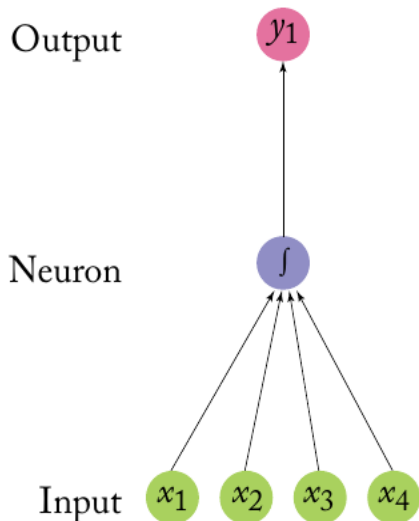
Neural Nets: Basics

- ▶ Neural networks $\leftrightarrow$ deep learning models
 - ▶ solve machine learning problems, just like logistic regression or gradient boosted machines
 - ▶ use tensorflow, torch, or huggingface, rather than sklearn or xgboost.
- ▶ **why use neural nets?**
 - ▶ greatly outperform standard ML techniques on specialized problems, for example language modeling

Neural Nets: Basics

- ▶ Neural networks $\leftrightarrow$ deep learning models
 - ▶ solve machine learning problems, just like logistic regression or gradient boosted machines
 - ▶ use tensorflow, torch, or huggingface, rather than sklearn or xgboost.
- ▶ **why use neural nets?**
 - ▶ greatly outperform standard ML techniques on specialized problems, for example language modeling
- ▶ **why not use neural nets?**
 - ▶ usually worse than standard ML on standard problems
 - ▶ outputs are a black box and difficult to interpret
 - ▶ models are often more challenging/labor-intensive to implement
 - ▶ computational constraints: training requires specialized hardware

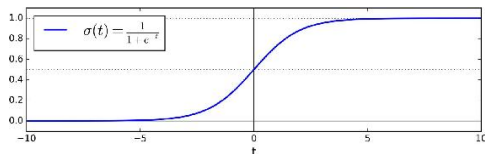
A “Neuron”



- ▶ applies dot product to vector of numerical inputs:
 - ▶ multiplies each input by a learned weight (parameter or coefficient)
 - ▶ sums these products
- ▶ applies a non-linear “activation function” to the sum
 - ▶ (e.g., the $\int$ shape indicates a sigmoid transformation)
- ▶ passes the output.

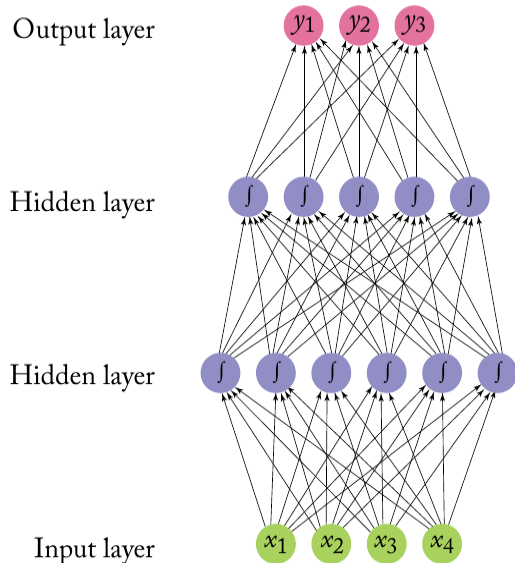
Logistic Regression $\approx$ “Neuron”

$$\hat{y} = \text{sigmoid}(\mathbf{x} \cdot \theta) = \frac{1}{1 + \exp(-\mathbf{x} \cdot \theta)}$$



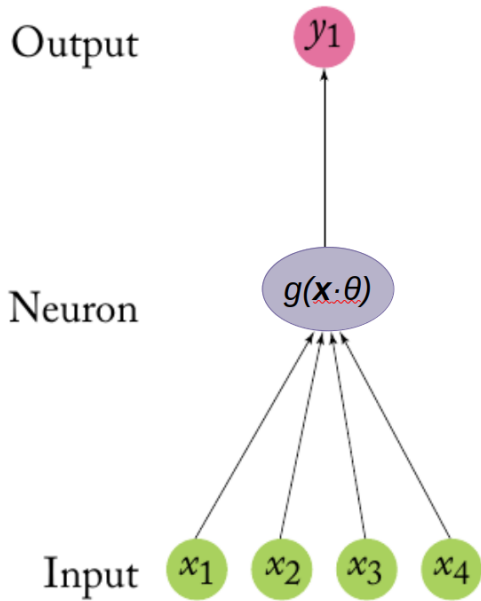
- ▶ applies dot product to vector of numerical inputs:
 - ▶ multiplies each input by a learned weight (parameter or coefficient)
 - ▶ sums these products
- ▶ applies a non-linear “activation function” (sigmoid) to the sum
- ▶ passes the output.

Multi-Layer Perceptron (MLP)



- ▶ A multilayer perceptron (also called a feed-forward network or sequential model) stacks neurons horizontally and vertically.
- ▶ alternatively, think of it as a stacked ensemble of logistic regression models.
- ▶ this vertical stacking is the “deep” in “deep learning”!

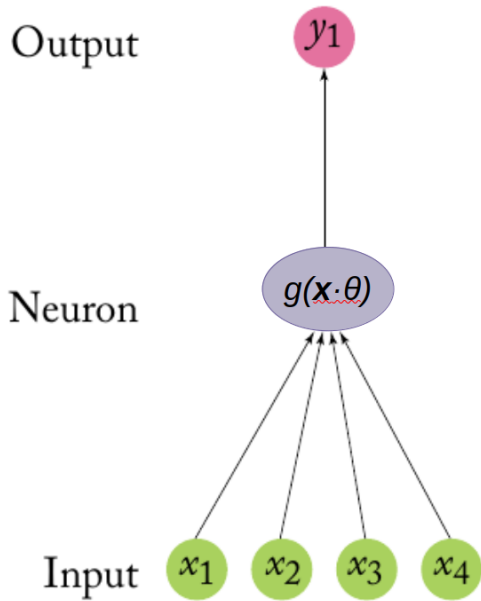
Activation functions $g(\mathbf{x} \cdot \theta)$



Previously we had

$$g(\mathbf{x} \cdot \theta) = \text{sigmoid}(\mathbf{x} \cdot \theta) = \frac{1}{1 + \exp(-\mathbf{x} \cdot \theta)}$$

Activation functions $g(\mathbf{x} \cdot \theta)$

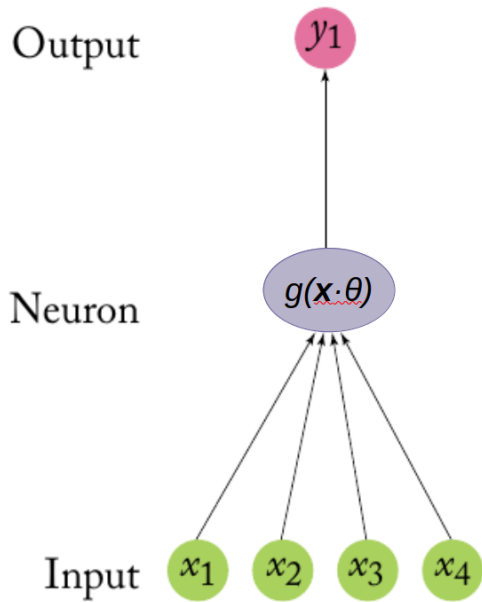


Previously we had

$$g(\mathbf{x} \cdot \theta) = \text{sigmoid}(\mathbf{x} \cdot \theta) = \frac{1}{1 + \exp(-\mathbf{x} \cdot \theta)}$$

But – it turns out that sigmoid does not work well in hidden layers, mainly because gradient is flat except around zero.

Activation functions $g(\mathbf{x} \cdot \theta)$



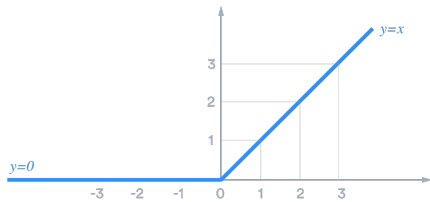
Previously we had

$$g(\mathbf{x} \cdot \theta) = \text{sigmoid}(\mathbf{x} \cdot \theta) = \frac{1}{1 + \exp(-\mathbf{x} \cdot \theta)}$$

But – it turns out that sigmoid does not work well in hidden layers, mainly because gradient is flat except around zero.

ReLU (rectified linear unit) function:

$$g(\mathbf{x} \cdot \theta) = \text{ReLU}(\mathbf{x} \cdot \theta) = \max\{0, \mathbf{x} \cdot \theta\}$$



Equation Notation: Multi-Layer Perceptron

- ▶ An multi-layer perceptron (MLP) with two hidden layers is

$$\mathbf{y} = \mathbf{g}_2(\mathbf{g}_1(\mathbf{x} \cdot \boldsymbol{\omega}_1) \cdot \boldsymbol{\omega}_2) \cdot \boldsymbol{\omega}_y$$

$$\mathbf{y} \in \{0,1\}^{n_y}, \mathbf{x} \in \mathbb{R}^{n_x}, \boldsymbol{\omega}_1 \in \mathbb{R}^{n_x \times n_1}, \boldsymbol{\omega}_2 \in \mathbb{R}^{n_1 \times n_2}, \boldsymbol{\omega}_y \in \mathbb{R}^{n_2 \times n_y}$$

- ▶ n_1, n_2 = dimensionality in first and second hidden layer.
- ▶ $\boldsymbol{\omega}_1, \boldsymbol{\omega}_2, \boldsymbol{\omega}_y$ = set of learnable weights for the first hidden, second hidden, and output layer
- ▶ $\mathbf{g}_1(\cdot), \mathbf{g}_2(\cdot)$ = element-wise non-linear functions (typically ReLU) for first and second layer.

Equation Notation: Multi-Layer Perceptron

- ▶ An multi-layer perceptron (MLP) with two hidden layers is

$$\mathbf{y} = \mathbf{g}_2(\mathbf{g}_1(\mathbf{x} \cdot \boldsymbol{\omega}_1) \cdot \boldsymbol{\omega}_2) \cdot \boldsymbol{\omega}_y$$

$$\mathbf{y} \in \{0,1\}^{n_y}, \mathbf{x} \in \mathbb{R}^{n_x}, \boldsymbol{\omega}_1 \in \mathbb{R}^{n_x \times n_1}, \boldsymbol{\omega}_2 \in \mathbb{R}^{n_1 \times n_2}, \boldsymbol{\omega}_y \in \mathbb{R}^{n_2 \times n_y}$$

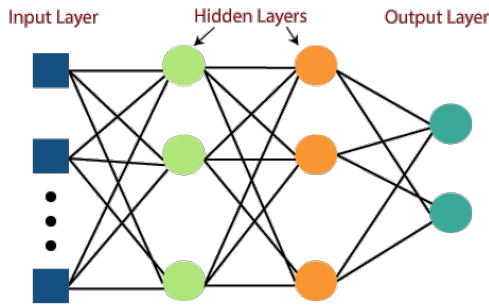
- ▶ n_1, n_2 = dimensionality in first and second hidden layer.
 - ▶ $\boldsymbol{\omega}_1, \boldsymbol{\omega}_2, \boldsymbol{\omega}_y$ = set of learnable weights for the first hidden, second hidden, and output layer
 - ▶ $\mathbf{g}_1(\cdot), \mathbf{g}_2(\cdot)$ = element-wise non-linear functions (typically ReLU) for first and second layer.
- ▶ Can also be written in decomposed notation:

$$\mathbf{h}_1 = \mathbf{g}_1(\mathbf{x} \cdot \boldsymbol{\omega}_1)$$

$$\mathbf{h}_2 = \mathbf{g}_2(\mathbf{h}_1 \cdot \boldsymbol{\omega}_2)$$

$$\mathbf{y} = \mathbf{h}_2 \cdot \boldsymbol{\omega}_y$$

where $\mathbf{h}_l$ indicate hidden layers.



Steps of NN Training (1/2)

- ▶ **Initialize Parameters:** Set weights and biases for all layers, typically randomly.
- ▶ **Input Forward Propagation:**
 - ▶ Pass input through (fully connected) layers. (For image analysis: CNNs with convolutional layers)
 - ▶ Apply activation functions (e.g., ReLU, Softmax).
- ▶ **Loss Calculation:** Compare predictions with true labels using a loss function (e.g., cross-entropy).

Steps of NN Training (2/2)

- ▶ **Backward Propagation:**
 - ▶ Compute gradients of the loss with respect to weights and biases using backpropagation.
 - ▶ Update weights through all layers via chain rule (from output to input).
- ▶ **Weight Update:** Adjust weights and biases using optimization algorithms (e.g., stochastic gradient descent).
- ▶ **Repeat:**
 - ▶ Iterate over the dataset for multiple epochs.
 - ▶ Evaluate on validation data to monitor performance and prevent overfitting.
- ▶ **Stop:** End training based on stopping criteria (e.g., convergence, maximum epochs).

Neural nets come with many hyperparameters

- ▶ **Epochs/iterations:** Number of times the model trains over the full set of images (e.g., 100 epochs)
- ▶ **Learning rate:** How much weights/coefficients change in optimization (e.g., 0.01)
- ▶ **Dropout rate:** To avoid overfitting; share of weights set to 0 (e.g., 0.5)
- ▶ **Train-validation ratio:** Split of images into sets (e.g., 80-20 split)
- ▶ **Loss function:** Evaluates model accuracy (e.g., Mean Squared Error)
- ▶ **Activation functions:** Nonlinear transformations (e.g., ReLU)
- ▶ More hyperparameters can be set (batch size etc.)

Example: NN for predicting outcomes (1/3)

- Show low-dimensional toy example predicting outcomes (vote shares) using county characteristics; can easily be adapted to high dimensional text case

County	White (%)	College (%)	Median Income (per capita, in hundreds USD)	Vote Share
Lake	87.70	16.20	215.37	0.46
Shasta	88.50	18.80	236.70	0.48
Mendocino	86.30	22.00	233.06	0.36
Sonoma	87.40	32.20	328.35	0.51
Sutter	74.00	18.70	236.02	0.55
Amador	90.70	19.30	273.47	0.52
Napa	84.80	31.30	347.95	0.60

Figure: Vote share for Hillary Clinton in the 2016 Democratic primaries in seven California counties (Webb Williams et al., 2020)

Example: NN for predicting outcomes (2/3)

$$\begin{array}{c}
 \text{(Input Layer: } 7 \times 4\text{)} \\
 X \\
 \begin{bmatrix} 1 & 87.7 & 16.2 & 215.37 \\ 1 & 88.5 & 18.8 & 236.70 \\ 1 & 86.3 & 22.0 & 233.06 \\ 1 & 87.4 & 32.2 & 328.35 \\ 1 & 74.0 & 18.7 & 236.02 \\ 1 & 90.7 & 19.3 & 273.47 \\ 1 & 84.8 & 31.3 & 347.95 \end{bmatrix}
 \end{array}
 \times
 \begin{array}{c}
 \text{(Weights: } 4 \times 2\text{)} \\
 \beta_1 \\
 \begin{bmatrix} 0.4634 & -0.3303 \\ -0.1495 & 0.1094 \\ -0.6841 & -0.0567 \\ 0.1183 & -0.3340 \end{bmatrix}
 \end{array}
 =
 \begin{array}{c}
 \text{(Hidden Layer 1 nontransformed: } 7 \times 2\text{)} \\
 Z_0 \\
 \begin{bmatrix} 1.7436 & -63.6019 \\ 2.3684 & -70.7870 \\ 0.0775 & -69.9929 \\ 4.2065 & -102.2821 \\ 4.5243 & -72.1398 \\ 6.0468 & -82.8577 \\ 7.5294 & -109.0627 \end{bmatrix}
 \end{array}$$

$$\begin{array}{c}
 \text{(Hidden Layer 1 transformed)} \\
 Z_1 \\
 \begin{bmatrix} 1.7436 & 0.0000 \\ 2.3684 & 0.0000 \\ 0.0775 & 0.0000 \\ 4.2065 & 0.0000 \\ 4.5243 & 0.0000 \\ 6.0468 & 0.0000 \\ 7.5294 & 0.0000 \end{bmatrix}
 \end{array}$$

$\max(0, Z_0) =$

Figure: An artificial neural network predicting Clinton's vote share in the 2016 Democratic primaries in seven California counties ($\hat{Y}$) as a function of percentage of white population (x_1), people with college education (x_2), the median income per capita (x_3), and an intermediate representation (Z_1): (1) using the input layer X to create a new hidden layer Z_0 , (2) applying a nonlinear ReLu transformation to the hidden layer Z_0 , ...

Example: NN for predicting outcomes (3/3)

(Hidden Layer 1 transformed: 7×3)

$$\begin{matrix} Z_1 \\ \begin{bmatrix} 1 & 1.7436 & 0.0000 \\ 1 & 2.3684 & 0.0000 \\ 1 & 0.0775 & 0.0000 \\ 1 & 4.2065 & 0.0000 \\ 1 & 4.5243 & 0.0000 \\ 1 & 6.0468 & 0.0000 \\ 1 & 7.5294 & 0.0000 \end{bmatrix} \end{matrix}$$

(Weights: 3×1)

$$\times \begin{matrix} \beta_2 \\ \begin{bmatrix} -0.4142 \\ 0.1084 \\ -0.1024 \end{bmatrix} \end{matrix} =$$

(Output Layer nontransformed: 7×1)

$$\begin{matrix} \hat{Y}_0 \\ \begin{bmatrix} -0.2252 \\ -0.1575 \\ -0.4058 \\ 0.0418 \\ 0.0762 \\ 0.2412 \\ 0.4019 \end{bmatrix} \end{matrix}$$

(Output Layer transformed)

$$\frac{1}{(1 + e^{-\hat{Y}_0})} = \begin{matrix} \hat{Y}_1 \\ \begin{bmatrix} 0.4420 \\ 0.4586 \\ 0.3979 \\ 0.5076 \\ 0.5170 \\ 0.5576 \\ 0.5962 \end{bmatrix} \end{matrix}$$

Figure: ... (3) using the features in the hidden layer to generate a set of predictions ($\hat{Y}_0$), and (4) applying a sigmoid transformation to these predictions to improve model fit ($\hat{Y}_1$) (Webb Williams et al., 2020).

Outline

Words Embedding with Local Contexts

Properties of Word Embeddings

How do I use Word Embeddings?

Neural Networks

Bias in Language

Implicit attitudes

"Attitudes that affect our understanding, actions, and decisions in an unconscious manner" (Kirnan institute, OSU)

Implicit attitudes

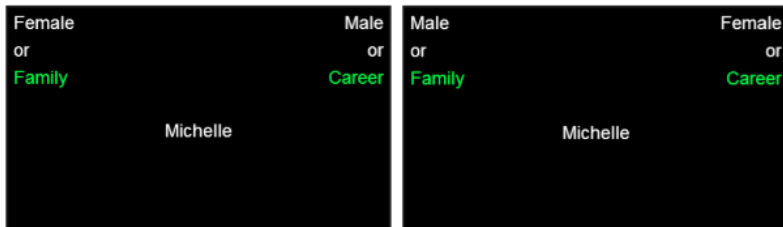
"Attitudes that affect our understanding, actions, and decisions in an unconscious manner" (Kirnan institute, OSU)

- ▶ Generally measured using Implicit Association Tests (IATs)

Implicit attitudes

"Attitudes that affect our understanding, actions, and decisions in an unconscious manner" (Kirnan institute, OSU)

- ▶ Generally measured using Implicit Association Tests (IATs)
- ▶ Subjects asked to assign words to categories (Greenwald et al. 1998)

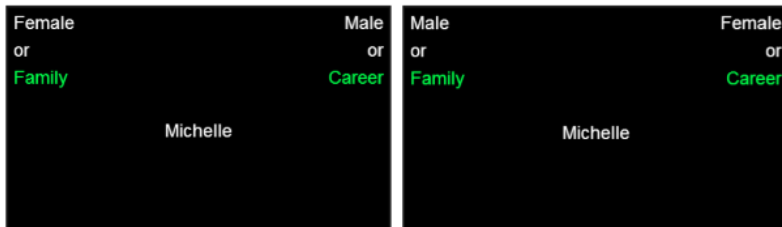


- ▶ Comparing reaction times across trials with different word pairs:
 - ▶ subjects tend to be slower and more error-prone in assignments against stereotype (e.g. "Michelle" goes to "Female or Career").

Implicit attitudes

"Attitudes that affect our understanding, actions, and decisions in an unconscious manner" (Kirnan institute, OSU)

- ▶ Generally measured using Implicit Association Tests (IATs)
- ▶ Subjects asked to assign words to categories (Greenwald et al. 1998)

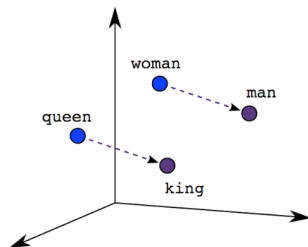


- ▶ Comparing reaction times across trials with different word pairs:
 - ▶ subjects tend to be slower and more error-prone in assignments against stereotype (e.g. "Michelle" goes to "Female or Career").
 - ▶ IAT score = difference in reaction time between stereotype-consistent and stereotype-inconsistent rounds.

Caliskan, Bryson, and Narayanan (*Science* 2017)

- ▶ “We replicated a spectrum of known biases, as measured by the Implicit Association Test, using a widely used, purely statistical machine-learning model trained on a standard corpus of text from the World Wide Web. . . .”

- ▶ “We replicated a spectrum of known biases, as measured by the Implicit Association Test, using a widely used, purely statistical machine-learning model trained on a standard corpus of text from the World Wide Web. . . .”

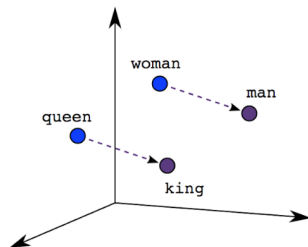


Analogies

- ▶ king : queen :: man : woman
- ▶ walked : walking :: swam : swimming

Caliskan, Bryson, and Narayanan (*Science* 2017)

- ▶ “We replicated a spectrum of known biases, as measured by the Implicit Association Test, using a widely used, purely statistical machine-learning model trained on a standard corpus of text from the World Wide Web. . . .”



Analogies

- ▶ king : queen :: man : woman
- ▶ walked : walking :: swam : swimming
- ▶ **man : programmer :: woman : homemaker**
- ▶ **he : physician :: she : nurse**

Example Stimuli

- ▶ Targets:
 - ▶ **Flowers:** aster, clover, hyacinth, marigold, poppy, azalea, crocus, iris, orchid, rose, bluebell, daffodil, lilac, pansy, tulip, buttercup, daisy, lily, peony, violet, carnation, gladiola, magnolia, petunia, zinnia.
 - ▶ **Insects:** ant, caterpillar, flea, locust, spider, bedbug, centipede, fly, maggot, tarantula, bee, cockroach, gnat, mosquito, termite, beetle, cricket, hornet, moth, wasp, blackfly, dragonfly, horsefly, roach, weevil.

Example Stimuli

- ▶ **Targets:**
 - ▶ **Flowers:** aster, clover, hyacinth, marigold, poppy, azalea, crocus, iris, orchid, rose, bluebell, daffodil, lilac, pansy, tulip, buttercup, daisy, lily, peony, violet, carnation, gladiola, magnolia, petunia, zinnia.
 - ▶ **Insects:** ant, caterpillar, flea, locust, spider, bedbug, centipede, fly, maggot, tarantula, bee, cockroach, gnat, mosquito, termite, beetle, cricket, hornet, moth, wasp, blackfly, dragonfly, horsefly, roach, weevil.
- ▶ **Attributes:**
 - ▶ **Pleasant:** caress, freedom, health, love, peace, cheer, friend, heaven, loyal, pleasure, diamond, gentle, honest, lucky, rainbow, diploma, gift, honor, miracle, sunrise, family, happy, laughter, paradise, vacation.
 - ▶ **Unpleasant:** abuse, crash, filth, murder, sickness, accident, death, grief, poison, stink, assault, disaster, hatred, pollute, tragedy, divorce, jail, poverty, ugly, cancer, kill, rotten, vomit, agony, prison.

Results

- ▶ Pleasant vs. Unpleasant?
 - ▶ Flowers vs. Insects
 - ▶ Musical instruments vs. weapons.

Results

- ▶ Pleasant vs. Unpleasant?
 - ▶ Flowers vs. Insects
 - ▶ Musical instruments vs. weapons.
 - ▶ European-American names vs. African-American names

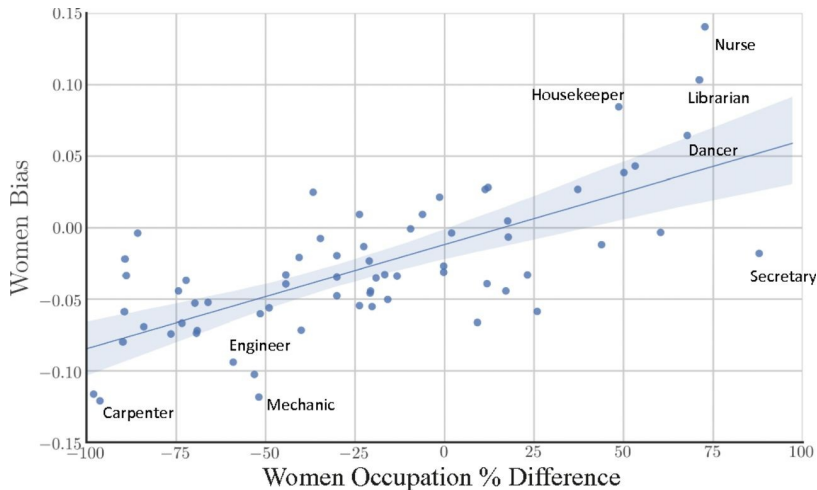
Results

- ▶ Pleasant vs. Unpleasant?
 - ▶ Flowers vs. Insects
 - ▶ Musical instruments vs. weapons.
 - ▶ European-American names vs. African-American names
- ▶ Male names vs. Female names:

Results

- ▶ Pleasant vs. Unpleasant?
 - ▶ Flowers vs. Insects
 - ▶ Musical instruments vs. weapons.
 - ▶ European-American names vs. African-American names
- ▶ Male names vs. Female names:
 - ▶ Career words (e.g. professional, corporation, ...) vs. family words (e.g. home, children, ...)
 - ▶ Math/science words vs arts words

Garg, Schiebinger, Jurafsky, and Zou (PNAS 2018)



Women's occupation relative percentage vs. embedding bias in Google News vectors.